

## บทที่ 2

# ทฤษฎีที่เกี่ยวข้อง

### 2.1 บทนำ

ทฤษฎีที่เกี่ยวข้องในการออกแบบและพัฒนาเกมสามมิติประเภท Simulation ในรายงานฉบับนี้ แบ่งออกเป็นสองด้าน ได้แก่

#### 2.1.1 ทฤษฎีการสร้างภาพสามมิติเบื้องต้น

เพื่อให้เข้าใจและสามารถสร้างฉากและองค์ประกอบต่าง ๆ ภายในเกมที่เป็นสามมิติได้

#### 2.1.2 ทฤษฎีเกี่ยวกับการเขียน AI ให้กับเกม

เพื่อนำไปสร้างให้ตัวละครภายในเกมที่ไม่ใช่ตัวละครที่เราบังคับ (NPC: Non Player Character) สามารถคิดตัดสินใจกระทำการใด ๆ เองได้ ทำให้การเล่นเกมน่าสนใจหลากหลายมากยิ่งขึ้น

### 2.2 ทฤษฎีการสร้างภาพสามมิติทั่วไป

#### 2.2.1 โครงสร้างของโมเดลแบบต่างๆ

- โครงสร้างแบบ Mesh และ Poly เป็นโครงสร้างที่ประกอบขึ้นจากจุดและเส้นตรงที่เชื่อมต่อกันจนเกิดเป็นแผ่นระนาบขึ้นมา และเมื่อนำแผ่นระนาบมาต่อกันเป็นรูปทรงต่างๆก็จะเกิดเป็นชิ้นงาน 3 มิติ โครงสร้างแบบนี้ถือเป็นโครงสร้างที่นิยมใช้มากที่สุดในปัจจุบันเพราะความง่ายในการขึ้นโมเดลเป็นรูปทรงต่างๆและในปัจจุบันโครงสร้างแบบ Mesh และ Poly ยังถูกพัฒนาไปจนถึงขั้นที่สามารถขึ้นโมเดลที่เป็นรูปแบบฟรีฟอร์มได้ โครงสร้างแบบนี้เหมาะกับการนำไปสร้างโมเดลสำหรับการทำเกม 3 มิติเนื่องจากเกม 3 มิตินั้นต้องการการประมวลผลภาพที่เร็ว โมเดลที่มีขนาดของ Poly น้อยก็จะทำให้ประมวลผลภาพได้รวดเร็วขึ้น

- โครงสร้างแบบ Spline เป็นโครงสร้างที่มีส่วนประกอบเช่นเดียวกับ Mesh และ Poly แต่แตกต่างกันที่ลักษณะของเส้นที่สามารถดัดโค้งได้เป็นอิสระก่อนที่จะสร้างพื้นผิว ซึ่งเหมาะสำหรับการสร้างรูปทรงแบบฟรีฟอร์ม

- โครงสร้างแบบ NURBS คล้ายกับโครงสร้างแบบ Spline แต่มีความยืดหยุ่นมากกว่าเหมาะสำหรับการสร้างรูปทรงแบบฟรีฟอร์ม

#### 2.2.2 ส่วนประกอบของโมเดล 3 มิติ

- Vertex คือ จุด (point) ซึ่งวางเรียงกันเป็นรูปทรงต่างๆ

- Vertices คือ เซตของจุดที่ใช้แทนตำแหน่งของวัตถุ เช่น Face หรือ Mesh ใน 3D Space โดยที่แต่ละ Vertex อาจจะแทนด้วยค่า 3 ค่าซึ่งมีลักษณะเฉพาะของจุดหรือ coordinate ในรูปทรง 3 มิติ

- Edge คือ เส้นที่เชื่อมต่อระหว่าง Vertex หนึ่งไปยังอีก Vertex หนึ่ง

- Polygon คือ ระนาบที่เกิดจากการนำ Edge มาวางเรียงต่อกัน โดยที่ 1 Polygon จะต้องมีย่าน้อย 3 Edge การทำภาพที่มีลักษณะโค้งจะใช้หลายๆ Polygon มาเรียงต่อกันให้ดูโค้งแทนการทำให้เป็นเส้นโค้งจริงเพื่อประหยัดเวลาในการคำนวณ ภาพ 3 มิติที่มีลักษณะโค้งที่มีจำนวนของ Polygon มากจะทำให้ภาพมีความโค้งมนสมจริงมากขึ้น แต่ก็ต้องใช้เวลาในการประมวลผลภาพมากยิ่งขึ้น

- Face คือ ส่วนประกอบที่อยู่ใน Mesh หรือ Poly ที่ถูกแบ่งครั้งนั่นเอง หรือก็คือ Vertices ที่มีตั้งแต่ 3 จุดขึ้นไปมาเชื่อมต่อกันเป็นรูปทรงต่างๆในแนวระนาบ Vertex เป็นตัวกำหนดมุมของ Face ทำให้ทุกๆ Vertex ใน Face จะต้องถูกกำหนดให้อยู่ในแนวระนาบ

- Mesh เกิดจากการรวมกันของ Face ที่เชื่อมต่อกัน ซึ่ง 1 Mesh สามารถมี Face ได้ตั้งแต่ 1 Face ขึ้นไป การรวมของ Face นี้ทำให้ง่ายต่อการจัดการวัตถุในการทำ Animation, Material และ Texture ชนิดการรวมกันของ Face มีดังนี้คือ

- Fan คือ กลุ่มของรูปทรงสามเหลี่ยม ซึ่งทุกรูปมีการใช้ Vertex ร่วมกัน 1 จุด โดยการกำหนดให้ Vertex นั้นๆอยู่ระหว่างกลางของสามเหลี่ยมเหล่านั้นชนิดของการรวม Face แบบ Fan นี้คล้ายกับแบบ Strip คือมีการกำหนดค่า Vertices 3 ค่าแรกสำหรับสามเหลี่ยมแรก จากนั้นในสามเหลี่ยมต่อไปก็เพียงแค่เพิ่มขึ้นมารูปละ 1 Vertex เท่านั้น

- Strip คือ กลุ่มของสามเหลี่ยม ซึ่งแต่ละรูปจะมีการใช้เส้นร่วมกันกับสามเหลี่ยมส่วนหน้าซึ่งหมายความว่า หลังจากที่มีการกำหนดค่า Vertices 3 ค่าแรกสำหรับสามเหลี่ยมแรก จากนั้นในสามเหลี่ยมต่อไปก็เพียงแค่เพิ่มขึ้นมารูปละ 1 Vertex นั่นเอง

- List คือ กลุ่มของสามเหลี่ยม ซึ่งทุกรูปไม่มีการใช้เส้นหรือ Vertex ร่วมกันเลย กล่าวคือ ค่า 3 ค่าของสามเหลี่ยมทุกรูปต้องกำหนดเองทั้งหมด

### 2.2.3 Texture (ลวดลายของพื้นผิว)

- Shading คือการทำรายละเอียดของตัวพื้นผิว เช่น ความมันวาว การสะท้อนของพื้นผิว หรือความโปร่งแสง ทึบแสงของวัตถุ เป็นการให้สีเป็นลำดับชั้น

- Flat Shading Lighting เป็นการรายละเอียดพื้นผิวที่มีสีเสมอกันทั่วทั้ง polygon

- Vertex Shading หรือ Gouraud Shading เป็นการให้สีแก่ vertex แต่ละจุดตามสีที่ได้กำหนดเอาไว้แล้ว

- Phong Shading มีลักษณะคล้ายกับ Gouraud Shading แต่จะให้แสงที่นวลกว่า แต่ก็ใช้เวลาในการ render นานกว่าด้วย

- Texture คือลวดลายของพื้นผิว โดยที่จะเป็น Bitmap ที่เป็น Pattern หรือ Image มักจะเก็บในรูปแบบไฟล์ BMP, PCX หรือ GIF เพื่อเป็นการใส่รายละเอียดให้แก่พื้นผิวของวัตถุ ทำให้วัตถุมีความสมจริง

- Texture Coordinate ใช้กำหนดการเชื่อมต่อกันระหว่าง Vertices ของ Face กับ Pixel ของ Bitmap โดย Texture Coordinate นี้ใช้แทน 2 มิติของ Coordinate System

- Texture Mapping คือ การวาดรูปลงบนพื้นผิวของ Face หรือ Polygon และในการทำ Texture Mapping นี้ต้องคำนึงถึงการคำนวณค่าต่างๆด้วย จึงต้องมีการกำหนดค่าของ Vertices ด้วย จากการที่ Texture Coordinate กำหนด Pixel ของ Texture ที่จะวาดลงในส่วนของ Face แล้วก็จะมีการ Wrapping เพื่อ Generate Texture Coordinate สำหรับ Object นั้นซึ่งการ Wrapping นั้นมี 4 ชนิดคือ

- Flat Warp จะทำการวาดลงบน Face โดยตรง

- Cubical Warp จะทำการ Warp Texture ใน Cube รอบๆ Object เหมือนกับการปะลวดลายบนวัตถุที่มีลักษณะเป็นกล่อง

- Cylindrical Warp จะทำการ Warp Texture ในทรงกระบอกรอบๆ Object ก็จะนำลวดลายในการ Map รอบโมเดลในลักษณะการห่อเหมือนทรงกระบอก ซึ่งเหมาะกับวัตถุที่มีลักษณะเป็นทรงกระบอกหรือเป็นแท่งๆ

- Spherical Warp จะทำการ Warp Texture ใน Sphere รอบๆ Object ก็จะนำภาพมาปะในลักษณะห่อรอบทรงกลม เหมาะกับการปะลวดลายบนวัตถุทรงกลม

- Shrink Wrap คล้ายกับการ Map แบบ Spherical แต่โปรแกรมจะรวมจุดปลายของลวดลายที่ นำมา Map เข้าหากันเป็นจุดเดียว

#### 2.2.4 Light (แสง)

แสงทำให้เกิดการเปลี่ยนแปลงของสีของ Vertices โดย Module ทำให้เกิด Vertex Normal เพราะสิ่งนี้ขึ้นอยู่กับมุมของแหล่งกำเนิดแสงตามปกติจะมีแสงสีขาว เพราะเป็น การรวมกันอย่างหนาแน่นของสีทุกสีและโดยมากมักใช้รูปแบบของ RGB ในการกำหนดสีของแหล่งกำเนิดแสง

ใน 3D นั้นมีการตั้งค่า RGB ของแม่สีต่างๆดังนี้

แสงสีขาว เป็น 1, 1, 1

แสงสีแดง เป็น 1, 0, 0

แสงสีน้ำเงิน เป็น 0, 0, 1

นอกจากนี้เรายังสามารถใช้ 3 สี นี้ในการผสมสีได้ ซึ่งแหล่งกำเนิดแสงมี 4 ชนิด ได้แก่

- Ambient Light คือแหล่งกำเนิดแสงที่ง่ายที่สุด เพราะไม่ต้องมีการกำหนดตำแหน่งของแหล่งกำเนิดแสง และยังให้ความสว่างทั่วทุก Object

- Point Light เป็นแหล่งกำเนิดแสงที่ทำการกระจายแสงไปทุกทิศทาง แต่ต้องระบุตำแหน่งของแหล่งกำเนิดแสง โดยไม่ต้องกำหนดทิศทางของแสง
- Directional Light เป็นแหล่งกำเนิดแสงที่มีประสิทธิภาพมากที่สุดเพราะ เป็นแหล่งกำเนิดแสงที่มีทิศทาง โดยต้องระบุตำแหน่งของแหล่งกำเนิดแสง
- Spot Light เป็นแหล่งกำเนิดแสงที่ต้องมีการระบุทั้งทิศทางและตำแหน่งของแหล่งกำเนิดแสง โดยการผลิตแสงจะเป็นรูปร่าง

### 2.2.5 Camera

Camera คือ มุมกล้องที่ทำให้เราสามารถมองเห็นวัตถุจากทิศทางต่างๆ เปรียบเสมือนว่าเรากำลังนำกล้องไปตั้งไว้บริเวณพื้นที่ที่มีวัตถุตั้งกล่าว ดังนั้นการเปลี่ยนมุมกล้องก็จะทำให้ภาพที่ปรากฏบนจอมีลักษณะแตกต่างกันไป การสร้างวัตถุเพียงชิ้นเดียวอาจจะไม่มีปัญหาอะไรในการมองรูปภาพ แต่ถ้าเราสร้างวัตถุหลายๆชิ้นหลายๆรูปทรงใน 1 ภาพ การลำดับตำแหน่งของวัตถุจึงเป็นสิ่งจำเป็นที่เราจะต้องฝึกให้คุ้นเคยและสามารถมองภาพให้เป็นมิติและสมจริงได้ ไม่ว่าภาพนั้นจะแสดงผลในรูปแบบใด ดังนั้นลำดับเป็นเรื่องที่เราต้องแยกแยะให้ออก หากว่าเราไม่สามารถที่จะมองภาพที่เราสร้างขึ้นให้เป็นมิติได้ ก็ยากที่จะรู้ได้ว่าภาพหรือวัตถุชิ้นไหนอยู่ข้างหน้าหรืออยู่ข้างหลัง และอาจทำให้เราเกิดความสับสนในเรื่องของการลำดับวัตถุ และทำให้การสร้างภาพหรือวัตถุมีความยุ่งยากซับซ้อนมากยิ่งขึ้น จนไม่สามารถสร้างเกมที่มีความสมบูรณ์และมีประสิทธิภาพได้

### 2.2.6 Device

Device คือ Object ที่สร้างขึ้นเพื่อใช้ในการวาดภาพจากมุมกล้องโดย Device มี 2 ชนิดหลักๆ คือ

- Software Device ซึ่งจะอนุญาตให้โปรแกรมสามารถวาดภาพโดยไม่ต้องนำไปคำนวณที่การ์ดจอแบบ 3D Accelerated
- Hardware Device สามารถทำงานบนคอมพิวเตอร์ที่มี 3D Hardware เท่านั้น และจะทำให้ Direct3D สามารถเรียกใช้คุณสมบัติต่างๆในการเรนเดอร์ภาพได้อย่างเต็มที่ ทำให้จำนวนเฟรม/วินาทีมีค่ามากขึ้น

### 2.2.7 Viewport

อยู่ในส่วนของ Camera โดย Viewport นั้นจะรวมไปถึงตำแหน่งและทิศทางของ Scene จากการมองเห็นซึ่งเรามักใช้ Viewport ร่วมกับ Field of View Front & Back Clipping และ Perspective Transformation

Viewport ประกอบด้วย

- Eyepoint คือ จุดที่ตั้งกล้อง

- Lookat คือ จุดที่กล้องมอง
- Upvector คือ Vector ด้านบนของกล้อง

การ View ดู Object มี 2 ชนิด

- Perspective เป็นวิธีที่ทำให้รูปที่วาดลงบน Screen มีลักษณะที่ใกล้เคียงกับ Object จริงๆ มากที่สุดเพราะจะสนใจทั้งจุด x, y, z ทำให้เกิดความลึกของภาพนั่นเอง
- Orthographic เป็นวิธีที่ทำให้รูปที่วาดลงบน Screen แบบง่ายๆ จึงจะเลยในส่วนของคุณค่า z ไปทำให้ภาพที่ได้คล้ายกับ 2 มิติ

## 2.2.8 Rendering

การ Render คือการสั่งให้โปรแกรมประมวลผลทุกสิ่งทุกอย่างไม่ว่าจะเป็น Material หรือ แสงเงาของภาพ 3 มิติให้ออกมาเป็นภาพ 2 มิติสำหรับการนำไปใช้งาน

## 2.2.9 Biped

Biped คือ ชุดโครงกระดูกสัตว์สองเท้า (มนุษย์) สำเร็จรูปที่ Character Studio ได้เตรียมเอาไว้ให้ใช้งานแทนใช้งานกระดูก (Bone) แบบเดิมๆ ที่มีใช้งานใน 3D Studio Max เมื่อเราต้องการให้ตัวละครของเราทำงานโดยอาศัยความสามารถของ Character Studio เราก็จะต้องใช้ Biped เป็นกระดูกให้กับตัวละครนั้นๆ ซึ่ง Biped เองนั้นจะช่วยให้เราลดขั้นตอนในการสร้างกระดูก เชื่อมต่อกระดูกชิ้นต่างๆ เข้ามาเป็นรูปร่างตัวละคร การทำงานที่เกี่ยวข้องกับการ Setup IK รวมทั้งการกำหนดชื่อให้กับกระดูกแต่ละชิ้นด้วย (ซึ่งบางท่านอาจจะไม่ให้ความสำคัญกับขั้นตอนการตั้งชื่อให้กับตัวละครเท่าไรนัก แม้แต่ตัวผมเองในบางโอกาสของการทำงานก็ตาม) ซึ่งการทำงานในขั้นตอนเหล่านี้เป็นเรื่องที่น่าเบื่อมากๆ โดยเฉพาะกับงานที่ต้องมีตัวละครมากๆ

ในส่วนของ Biped หนึ่งตัวจะประกอบด้วยชิ้นส่วนต่างๆ มากมายหลายชิ้นตามจำนวนชิ้นส่วนข้อต่อหลักของมนุษย์ในส่วนของสะโพกเราเห็นรูปสี่เหลี่ยมเล็กๆ ฝังอยู่ด้านใน (มองเห็นชัดใน Wireframe) ชิ้นส่วนนี้เรียกว่า Center of Mass ทำหน้าที่เป็นศูนย์กลาง (Root) ของ Biped

## 2.2.10 Animation

Keyframe Animation - เป็นที่ทราบกันทั่วไปว่า หลักการทำ Animation ขั้นพื้นฐานนั่นก็คือ การนำเอาภาพนิ่งที่ต่อเนื่องกันจำนวนมากมาเปิดทีละภาพต่อกันด้วยความเร็วสูง ภาพชุดนั้นๆ ก็จะดูเป็นภาพ เคลื่อนไหวขึ้นมาได้ ภาพแต่ละภาพที่ถูกเปิดขึ้นมาทีละภาพนั้นเรียกว่า

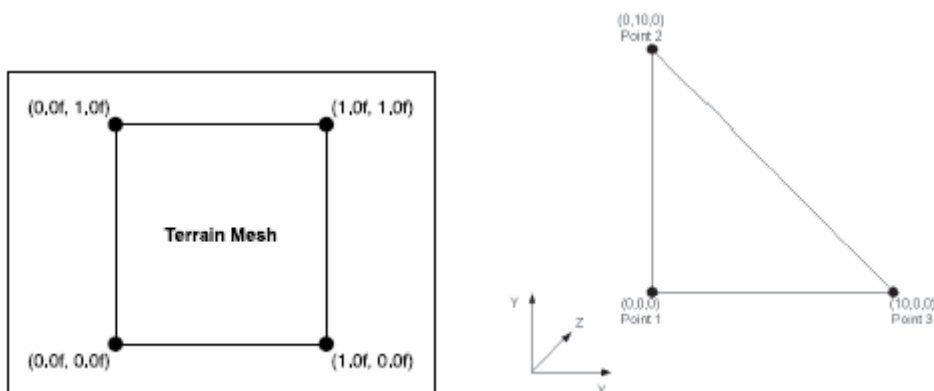
Frame โดยหน่วยที่ใช้วัดคุณภาพของ Animation ก็จะใช้นับจำนวนของภาพหรือ Frame ที่จะถูกเปิดขึ้นมาในช่วง เวลา 1 วินาที เช่น Animation แบบ 8 Frame ต่อวินาทีนั่นหมายถึง ในหนึ่งช่วงเวลา 1 วินาทีจะต้องใช้ภาพนิ่งจำนวน 8 ภาพ ซึ่งการทำ Animation ในลักษณะนี้จะเห็นว่ายุ่งยากมาก เพราะถ้าเป็น Animation เรื่องยาวๆ ก็เป็นอันต้องเขียนภาพจำนวนมากๆ อย่าง

หลีกเลี่ยงไม่ได้ แต่วิธีการทำ Animation แบบนี้ถือว่าเป็นวิธีพื้นฐานที่สุดแม้ทุกวันนี้การทำ Animation หลายๆ เรื่องก็ยังคงใช้การทำงานแบบนี้อยู่

สำหรับการทำงาน Animation ในงาน 3D นั้นก็ยังอาศัยหลักการเดียวกันกับการทำ Animation ที่ได้กล่าวมาแล้ว แต่ก็ได้มีการลดขั้นตอนการทำงานให้สั้นย่อสะดวกสบายยิ่งขึ้น เรียกว่าการทำ Animation แบบ In-Between Frame ซึ่งการทำ Animation ใน ลักษณะนี้เราจะกำหนดให้มี Frame บาง Frame ทำหน้าที่เป็นตัวบันทึกจังหวะในการเคลื่อนที่ในแต่ละช่วงเวลาของวัตถุใน Scene ซึ่งเราจะเรียก Frame ที่ทำหน้าที่บันทึกการเคลื่อนที่ของวัตถุนั้นว่า Key Frame สำหรับการทำ Animation ในแบบของการกำหนด Key Frame นั้นจะทำโดยการนำเอาตำแหน่งของในแต่ละ Key Frame มาคำนวณหาความเป็นไปได้ของตำแหน่งใน Frame ที่อยู่ ระหว่าง Key Frame แบบอัตโนมัติ ซึ่งการทำงาน ในลักษณะนี้จะประหยัดเวลากว่ามากสำหรับการทำ Animation แบบเดิมๆ โดยเฉพาะกับ Animation ที่มีความยาวมากๆ แต่กลับใช้คนน้อยๆ ในการทำงานซึ่งโปรแกรม คอมพิวเตอร์ที่ใช้ทำงานด้าน Animation เกือบจะทั้งหมดต่างก็ใช้หลักการทำ Animation แบบ Key Frame เป็นหลักทั้งสิ้น

### 2.2.11 กราฟิก 3 มิติทั่วไป

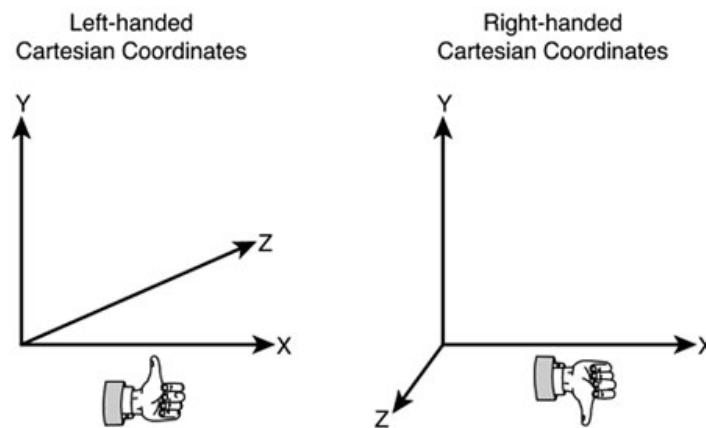
#### 2.2.11.1 โครงสร้างพื้นฐานของโมเดล 3D



รูปที่ 2.1 โพลีกอน 2D และ โพลีกอน 3D

โพลีกอน (Polygon) คือ รูปหลายเหลี่ยมโดยมีรูปทรงสามเหลี่ยมเป็นพื้นฐานที่ต่ำที่สุด ประกอบด้วย 3 จุด และจุดหรือ เวกเตอร์ (Vertex) คือ ส่วนย่อยที่เล็กที่สุดในระบบ 3 มิติ ซึ่งก็คือจุดที่ตั้งอยู่ภายในสเปซ 2 มิติหรือ 3 มิติ เราสามารถสร้างเส้นได้โดยการรวมสองเวกเตอร์เข้าด้วยกัน เส้นสามเส้นจะรวมกันเป็นโพลีกอน ความสัมพันธ์ของจุดเวกเตอร์ที่สามที่สามารถวาดเส้นเชื่อมต่อกันเป็นรูปร่าง คือ พื้นผิวโพลีกอน (Polygon face)

### 2.2.11.2 ระบบพิกัด 3 มิติ (3D Coordinate System)



รูปที่ 2.2 ระบบพิกัด 3 มิติ (3D Coordinate System)

ใน Direct3D จะใช้กฎหัวแม่มือซ้าย ซึ่งจะแตกต่างจาก OpenGL ซึ่งใช้กฎหัวแม่มือขวา

### 2.2.11.3 การทรานฟอร์ม (Transformation)

คือสิ่งที่ต้องการเปลี่ยนแปลงเวอร์เท็กซ์ของวัตถุ โดยทรานสเลชัน (translation) คือย้ายจุดกึ่งกลาง สเกล (scale) คือขยาย (หรือกระจายเวอร์เท็กซ์จากจุดกึ่งกลาง) โรเตด (rotate) คือหมุนเวอร์เท็กซ์รอบจุดกึ่งกลาง

3D translation  
transformation  
matrix

|       |       |       |   |
|-------|-------|-------|---|
| 1     | 0     | 0     | 0 |
| 0     | 1     | 0     | 0 |
| 0     | 0     | 1     | 0 |
| $T_x$ | $T_y$ | $T_z$ | 1 |

3D scaling  
transformation  
matrix

|       |       |       |   |
|-------|-------|-------|---|
| $S_x$ | 0     | 0     | 0 |
| 0     | $S_y$ | 0     | 0 |
| 0     | 0     | $S_z$ | 0 |
| 0     | 0     | 0     | 1 |

z-axis, 3D rotation  
transformation matrix

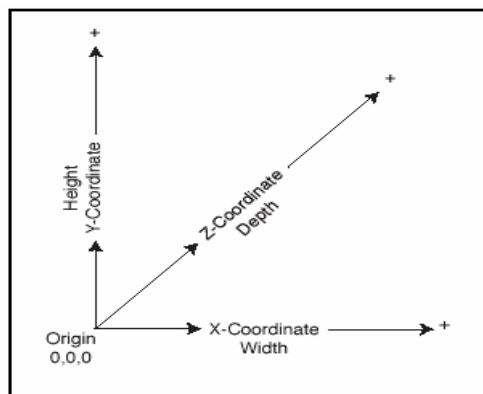
|           |          |   |   |
|-----------|----------|---|---|
| $\cos r$  | $\sin r$ | 0 | 0 |
| $-\sin r$ | $\cos r$ | 0 | 0 |
| 0         | 0        | 1 | 0 |
| 0         | 0        | 0 | 1 |

x-axis, 3D rotation  
transformation matrix

|   |           |          |   |
|---|-----------|----------|---|
| 1 | 0         | 0        | 0 |
| 0 | $\cos r$  | $\sin r$ | 0 |
| 0 | $-\sin r$ | $\cos r$ | 0 |
| 0 | 0         | 0        | 1 |

y-axis, 3D rotation  
transformation matrix

|          |   |           |   |
|----------|---|-----------|---|
| $\cos r$ | 0 | $-\sin r$ | 0 |
| 0        | 1 | 0         | 0 |
| $\sin r$ | 0 | $\cos r$  | 0 |
| 0        | 0 | 0         | 1 |



รูปที่2.3 รูปโครงสร้างเมทริกซ์ (Matrix)

หากต้องการทำทุกอย่างให้คุ้นกันตามลำดับ สเตจ โรเตดและทรานสเลชั่น

## 2.3 ทฤษฎีเกี่ยวกับการเขียน AI ให้กับเกม

### 2.3.1 Artificial Intelligence (AI)

AI (Artificial Intelligence) นั้นได้แน่วโน้มความคิดส่วนมากมาจากมนุษย์ คือ ต้องการจะสร้างตัว NPC (Non Player Character) ให้ออกมาให้คล้ายกับลำดับความคิดของมนุษย์มากที่สุด โดยมีหลักการความคิดได้หลายแบบ ซึ่งจะอธิบายหลักการคิดของ AI อย่างคร่าวๆ ดังนี้

- Expert systems เป็นการพยายามในการศึกษาเกี่ยวกับลำดับความคิด การตอบสนองของมนุษย์เป็นหลัก
- Case-based reasoning เป็นเทคนิคที่ใช้ในการวิเคราะห์ข้อมูล ที่ได้รับมาจากสภาพแวดล้อม และนำไปเปรียบเทียบกับความรู้เก่าหรือข้อมูลที่มีเก็บไว้ใน database และนำมาวิเคราะห์เหตุการณ์เก่าที่คล้ายคลึงกับเหตุการณ์ใหม่มากที่สุด เพื่อที่จะแสดงผลลัพธ์ออกมา เป็นเหมือนประสบการณ์ของมนุษย์
- Finite-state machines เป็นตัวอย่างง่ายๆ ที่มีไว้เพื่อหาว่ามีกี่ state และมีการติดต่อกันอย่างไร ระหว่าง state ใน Finite-state machines มีโอกาสที่ state หนึ่งสามารถไปได้อีกหลาย state โดยไม่ยึดติดกับ state ใด state หนึ่ง
- Production systems เป็น database ที่เก็บกฎเกณฑ์ต่างๆ ซึ่งกฎเกณฑ์ประกอบไปด้วยเงื่อนไข และการกระทำต่อเงื่อนไขนั้น เป็นเหมือนลำดับของ if-then ในการแก้ปัญหาที่มีความสลับซับซ้อนของลำดับเหตุการณ์ให้เหมาะสมที่สุด
- Decision trees คล้ายกับเงื่อนไขของ if-then โดยจะทำการตัดสินใจเลือก input ที่เป็นตัวแรกหรือ root node หรือตัวสุดท้ายของ node และ input ตัวถัดไปที่จะถูกเลือกเป็น child node โดยวิธีการนี้สามารถทำการสร้างเป็น tree ได้



- Search methods เป็นวิธีการที่เกี่ยวกับการหาลำดับในการกระทำหรือ state ต่างๆเพื่อไปหาจุดมุ่งหมาย หรือจุดสุดท้ายของ state
- Planning systems และ scheduling systems เป็นวิธีการที่ขยายมาจาก search methods โดยเน้นไปในเรื่องปัญหาย่อย ๆ เพื่อที่จะหาลำดับในการกระทำที่ดีที่สุด โดยทำการหา state เริ่มต้นที่แน่นอนและลำดับวิธีในแต่ละการกระทำที่เป็นไปได้
- First-order logic เป็นการทำให้ AI agent อยู่ภายในสภาพแวดล้อม โดยจะมีการกำหนดให้โลกทั้งหมดประกอบไปด้วย objects และ properties และ แสดงความสัมพันธ์กันระหว่าง objects และ properties เหล่านั้น
- Multi-agent systems เป็นการรวมพฤติกรรมตามธรรมชาติที่แสดงออกมาระหว่างหลายๆ agent ร่วมกัน
- Artificial life (A-life) พัฒนามาจาก Multi-agent systems ที่มีการประยุกต์ โดยจะแสดงคุณสมบัติทั่วไปที่เกี่ยวกับชีวิตความเป็นอยู่ของ AI agent ในโลกสมมติ
- Flocking เป็นส่วนหนึ่งของ A-life เป็นเทคนิคในการเคลื่อนย้าย AI agent ที่อยู่ร่วมกันในรูปแบบต่างๆที่สังเกตเห็นเหมือนกับชีวิตจริงเช่น ผีเสื้อ และ การจับกลุ่ม
- Robotic เป็นกลไกที่การทำงาน โดยมีการตอบโต้กับโลกของความเป็นจริง
- Genetic algorithms และ genetic programming เป็นเทคนิคในการพัฒนาแบบจำลองพันธุกรรม ด้วยวิธีการเลือก และการผสมพันธุ์ โดยการสุ่มแบบสลับกันหรือ เปลี่ยนไปจากเดิม
- Neural networks เป็นระดับโครงสร้างพื้นฐานของกลไกการเรียนรู้ ภายในการติดต่อกับระบบประสาทในสมองของสัตว์และระบบประสาทอื่น
- Fuzzy logic เป็นการใช้เลขจำนวนจริงในการแบ่งเป็นระดับของสมาชิกจำนวนในเซตตรงข้ามกับ Boolean (true or false) ซึ่งเป็นค่าของ traditional logic

### 2.3.2 AI ที่ใช้ในเกม

AI ในเกมทั่วไปเป็นแบบ search tree เช่น เกม chess ซึ่งเป็นเกมแบบแรกๆที่มีการสร้างเป็น AI หรือ path finding และได้มีการพัฒนาการให้ AI เกิดมีความรู้สึกและแสดงท่าทางในการตอบสนองที่คล้ายมนุษย์และสัตว์ และได้นำมาพัฒนาให้ดีขึ้นกว่าการใช้วิธีการสุ่มแบบเดิมของพฤติกรรมของ AI เพื่อให้แน่ใจว่าการเล่นเกมนั้นในแต่ละครั้งมีความแตกต่างกันทุกครั้ง

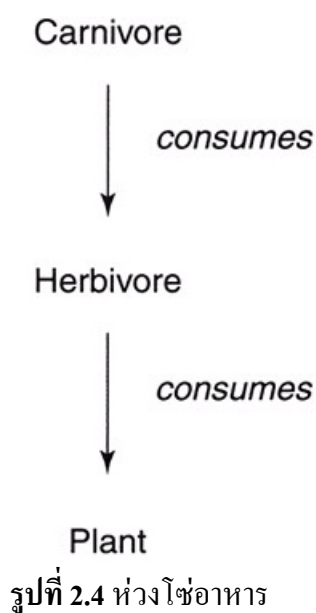
ในเกมประเภท Simulation หรือเกมที่เป็นการจำลองที่ได้ทำการศึกษาในโครงงานนี้จะมีการใช้รูปแบบของ AI คือมี Artificial life ("A-life") เป็นเทคนิคใน AI ที่ใช้ในการควบคุมพฤติกรรมของ AI agent และ multi-agent จะใช้การ Flocking ซึ่งในเกม The Sims จะมีประโยชน์ในการเคลื่อนไหวของ AI และยังมีการใช้ Basic Probability ซึ่งเป็นการใช้ความน่าจะเป็นมาช่วยให้เกมมีความไม่แน่นอน ไม่สามารถคาดเดาได้โดยง่าย

### 2.3.2.1.) A-life

เนื่องจากทฤษฎีของ A-life มีเยอะมาก ในที่นี้เราจึงต้องยกมาแค่บางส่วน โดยเราจะทำการยกตัวอย่างของ Synthetic ethnology ซึ่งเป็นการศึกษาพฤติกรรมของสัตว์ ที่มีพฤติกรรมและการพัฒนาการในโลกของการจำลอง และสัตว์เหล่านี้สามารถที่จะมีความนึกคิดระหว่างจุดประสงค์นั้นได้ หรือจะสมมุติเหตุการณ์บางอย่างได้

A-life สามารถอธิบายอย่างง่ายๆ โดยเราจะใช้สิ่งแวดล้อมในการเล่นโดยจะมี “what if” เป็นคีย์เวิร์ดในเกม ว่าระบบและสิ่งแวดล้อมจะมีการตอบสนองต่อการเปลี่ยนแปลงอย่างไร

ห่วงโซ่อาหารจะอธิบายเป็นลำดับรูปแบบของการมีชีวิตภายใน ecosystem ยกตัวอย่างง่ายๆ เช่น รูปแบบห่วงโซ่อาหารที่มีอยู่ 3 ระดับ ระดับล่างสุดจะเป็นพืช (Plant) ซึ่งพืชเหล่านี้ได้รับพลังงานมาจากสิ่งแวดล้อม เช่น แสงแดด เป็นต้น ระดับกลางจะเป็นสัตว์กินพืช (Herbivore) สัตว์กินพืชจะกินพืชเพื่อให้ชีวิตอยู่รอด สุดท้ายระดับบนสัตว์กินสัตว์ (Carnivore) จะทำการกินสัตว์เป็นอาหารเพื่อให้ชีวิตอยู่รอด ถ้ามีระดับใดตาย จะส่งผลถึงอีกระดับหนึ่งเสมอ



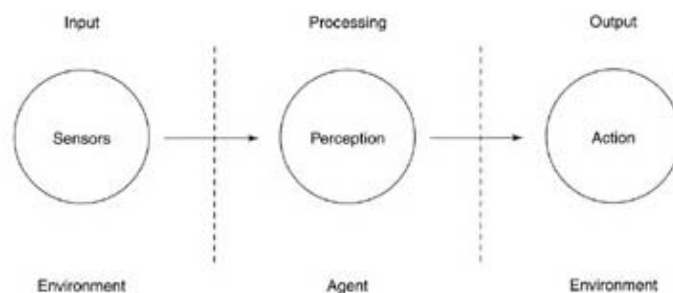
ถ้าสิ่งที่อยู่ในระดับใดระดับหนึ่งในห่วงโซ่อาหารเกิดการเปลี่ยนแปลงจะส่งผลถึงระดับอื่นด้วย เช่น ถ้าพืชเกิดการลดลง จะส่งผลต่อสัตว์กินพืชลดจำนวนลงด้วย และจะส่งผลต่อสัตว์กินสัตว์ต่อไปอีกเช่นกัน เพื่อทำให้ห่วงโซ่อาหารเกิดความสมดุล

ในห่วงโซ่อาหาร ตัว agent จะมีการรับเหตุการณ์และตอบสนองกับสิ่งแวดล้อมด้วยตัวมันเอง ในรูปแบบของห่วงโซ่อาหารอย่างง่ายที่แสดงให้ดูดังรูปที่ 2.4 พืชจะอยู่ทั่วไปตามสิ่งแวดล้อมและจะเป็นอาหารให้กับสัตว์กินพืชซึ่งเป็น agent ซึ่งตัว agent จะต้องทำการหาพืชโดยจะย้ายถิ่นไปเรื่อยๆ ส่วนผู้ล่า agent ซึ่งเป็นสัตว์กินสัตว์จะก็จะ

คอยย้ายถิ่นไปเรื่อยๆ เพื่อทำการล่าสัตว์กินพืช หรือ agent ถ้ามีตัว agent จำนวนมากจะส่งผลให้มี agent จำพวกหนึ่งอดตาย เมื่อ agent มีอาหารที่เพียงพอจะส่งผลให้มี agent ตัวใหม่เพิ่มขึ้นมาในสิ่งแวดล้อมนี้ และใน agent ที่เกิดขึ้นมาใหม่นี้จะเกิดการเปลี่ยนแปลง ทำให้เกิดการพัฒนารูปร่างทางสมอง

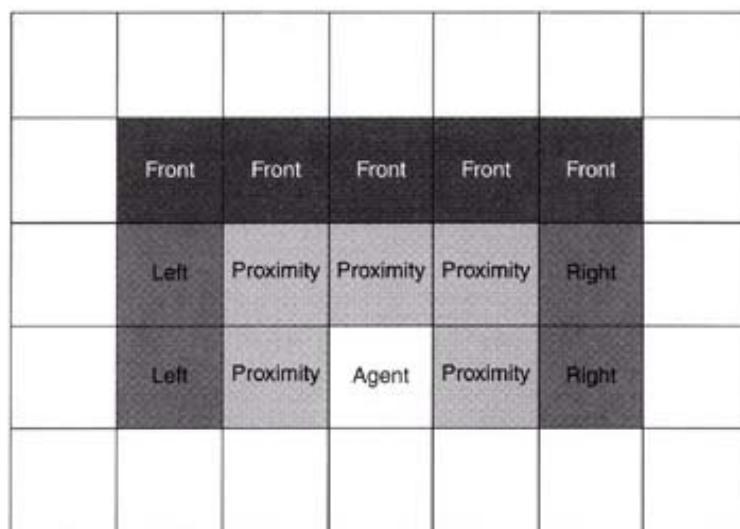
การที่ตัว agent จะอยู่รอดได้จะต้องทำการกินพืช และทำการหนีผู้ล่าคือสัตว์กินสัตว์ และกระบวนการเหล่านี้จะทำให้ตัว agent ได้รับประสบการณ์และการเรียนรู้ ส่งผลให้เกิดการพัฒนาการของตัว agent

ในกระบวนการทำงานของ agent นี้จะมีความทำงานโดยจะทำการมองเห็นสิ่งแวดล้อมและจะทำการเซตเป็น input หรือ perception และจะนำเอา perception เข้าไปประมวลผลเพื่อให้ได้ output ออกมาเพื่อตอบสนองกับสิ่งแวดล้อม ในการประมวลผลนี้ ตัว agent จะได้จากประสบการณ์และการเรียนรู้ หรือได้จากการถ่ายทอด ดังรูปที่ 2.5



รูปที่ 2.5 รูปแบบการทำงานของ agent

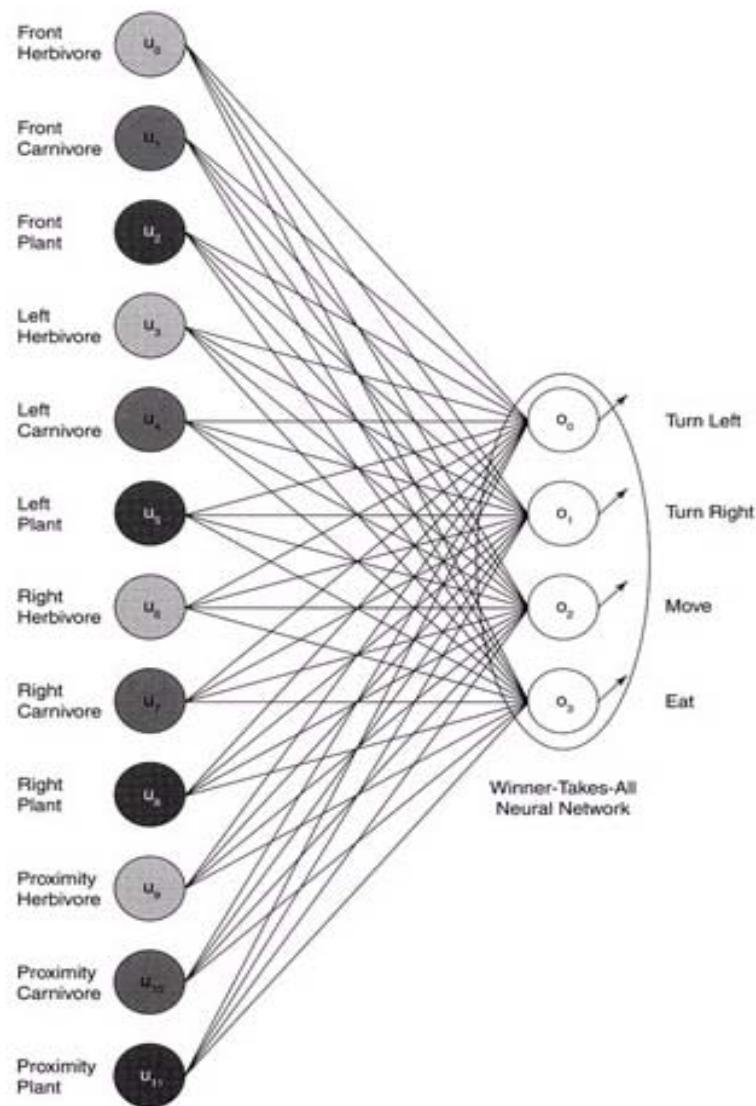
ใน Sensors ตัว agent จะมีความสามารถในการรับรู้ได้ในส่วนที่ตัวเองอยู่เท่านั้น และจะไม่สามารถรับรู้ได้ทั้งหมดในสิ่งแวดล้อม จากรูปที่ 2.6 จะแสดงให้เห็นรูปแบบการมองเห็นของตัว agent



รูปที่ 2.6 พื้นที่ perception ของ agent

พื้นที่ใกล้เคียงของตัว agent จะถูกเรียกว่า Proximity ซึ่งจะส่งผลต่อตัว agent มากกว่าตำแหน่งอื่น เพราะการกระทำที่ agent จะมีต่อสิ่งแวดล้อมจะทำได้แค่ในส่วนของ Proximity เท่านั้น

ภายในสมองของ agent จะมีการคำนวณมากมายมีลักษณะการจำลองแบบ state machines มีการแบ่งแยกเป็นระบบ หรือการใช้ neural networks จากรูปที่ 2.7 จะแสดงให้เห็นว่ากระบวนการคิดของ agent ซึ่งจะมีการติดต่อกันภายในและหาสิ่งที่ดีที่สุดมาตอบสนอง



รูปที่ 2.7 รูปแบบการคิดเพื่อหาสิ่งที่ดีที่สุด

จากรูปที่ 2.7 เราจะใช้น้ำหนักมาเป็นตัวช่วยในการคิดเพื่อเลือกสิ่งที่ดีที่สุดของแต่ละอันมาคิดดังสมการที่ 2.1

$$o_j = b_j + \sum_{i=0}^n u_i w_{i,j}$$

(2.1)

เมื่อ  $O$  เป็น output ที่จะแสดงออกไปของตัว agent

$U$  เป็น input ที่ได้มาจากการสังเกตสิ่งแวดล้อม

$W$  เป็น ค่าน้ำหนักที่ได้มาจากการทำซ้ำๆ หรือประสบการณ์

$b$  เป็น ค่าเริ่มต้นที่มีผลต่อ output

ตัว agent จะอยู่ในสิ่งแวดล้อมได้เมื่อมีพลังงานที่เพียงพอหรือ ไม่เป็นศูนย์ ถ้าหากตัว agent มีพลังงานเป็นศูนย์ตัว agent จะตาย agent จะต้องหาอาหารจากสภาพแวดล้อมที่มีอยู่ โดยตัว agent จะมีระดับความต้องการพลังงานซึ่งจะต้องรักษาระดับไม่ให้ลดลงจนเกินไป

ในการแพร่พันธุ์เมื่อ agent มีพลังงานมากกว่า 90% จะทำการแพร่พันธุ์โดยจะมีการถ่ายทอดประสบการณ์และความรู้โดยการสุมจากพ่อและแม่ของมัน และการแพร่พันธุ์จะครึ่งหนึ่งจะลดพลังงานของพ่อและแม่เพื่อไม่ให้เกิดการแพร่พันธุ์ต่อไปอีก

ระหว่างการจำลองนี้ จะเกิดการแข่งขันขึ้นเสมอโดย ตัว agent ต้องพยายามหนีสัตว์กินสัตว์และ สัตว์กินสัตว์จะพยายามล่า agent เพื่อความอยู่รอดเช่นเดียวกัน

ในการควบคุมพฤติกรรมของตัว agent ที่มีความซับซ้อน Steering behaviors เป็นอีกเครื่องมือหนึ่งเพื่ออธิบายลักษณะกริยาท่าทางของ agent เป็นกลไกระดับต่ำกว่าด้วยเรื่องการจำลองชีวิต ที่เกี่ยวกับการเคลื่อนที่ของตัวละครหนึ่ง รวมไปถึงสัตว์ต่างๆ

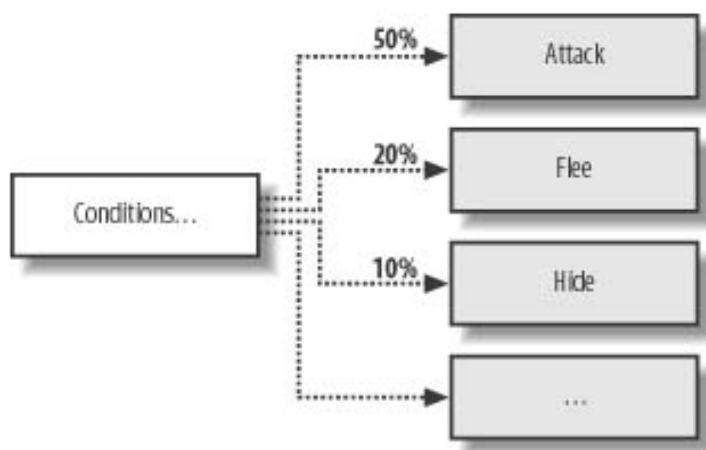
ปกติสภาพแวดล้อมภายในเกม จะมีความหมายที่ขัดแย้งกันอยู่ เป็นหัวใจหลักในการส่งผลต่อการกระตุ้นตัวละครต่างๆ ให้หยุด หรือกระทำการใด ๆ ลงไป จากความขัดแย้งกันนี้จะมีอยู่ทั่วไปในโลกจำลอง และจากวิธีการนี้ทำให้การเคลื่อนไหวของตัวละครเป็นอิสระจากวิธีการแบบเดิม

การค้นหาและการหลบหนีเป็น Steering behaviors แบบหนึ่งที่เป็นรูปแบบลำดับของการไปสู่จุดหมายที่ต้องการจะไป โดยมีการทำนายตำแหน่งจุดหมายปลายทางในอนาคต ในการค้นหาเป็นพฤติกรรมอย่างง่ายที่ต้องการหาจุดหมายปลายทาง ส่วนในทางตรงกันข้ามคือการหลบหนีซึ่งเป็นพฤติกรรมที่ต้องการหนีจากจุดหมาย วิธีที่ใช้ในการค้นหา ยกตัวอย่าง เช่น path following และ flocking

#### 2.3.2.2.) Basic Probability

เป็นการใช้ความน่าจะเป็นมาใช้กับหลักการของเกมที่จะให้ AI ไม่สามารถคาดเดาได้

- Randomness เป็นการสร้างเลขจำนวนในการสุ่มที่เป็นความน่าจะเป็น
- Character abilities เป็นการนำความน่าจะเป็นเพื่อนำมาทำการอธิบายคุณสมบัติของสิ่งต่างๆแต่ละชนิด เช่น นำมาอธิบายความเกี่ยวข้องกันของราศีกับความสนใจของแต่ละราศี ซึ่งเป็นคุณสมบัติอย่างหนึ่ง โดยที่อาจจะแสดงเป็นตารางได้
- State Transitions เป็นการทำสิ่งต่างๆในรูปแบบที่เป็นขั้นเป็นตอน โดยใช้ state transition ใน finite state machine กับ ความน่าจะเป็นผสมกัน ดังตัวอย่างในรูปที่ 2.8



รูปที่ 2.8 Creature states

**2.3.2.3.) Fuzzy Logic** มาจาก Fuzzy Set Theory ซึ่งใช้จัดการเกี่ยวกับการใช้เหตุผลที่ได้จากการประมาณแทนการอนุมานโดยตรงจากข้อสรุปพื้นฐานทางตรรกศาสตร์ ในบางครั้งมักจะนำเอา Fuzzy Logic ไปใช้กับความน่าจะเป็น แต่อย่างไรก็ตาม Fuzzy Logic มีหลักการที่ชัดเจน คือ ค่าความจริงของ Fuzzy ถูกแสดงอยู่ในเซตที่มีสมาชิกที่มีความกำกวม ซึ่งไม่ใช่ความเป็นไปได้ของเหตุการณ์หรือเงื่อนไขบางอย่างอย่างใดอย่างหนึ่งภายในเซตไม่ใช่การสุ่ม

ตัวอย่างเช่น ตัวละครในบทละครชื่อ Bob อาศัยอยู่ในบ้านที่มีห้องครัวและห้องรับประทานอาหารซึ่งอยู่ติดกัน หลายครั้งที่สถานะของ Bob อยู่ในเซตซึ่งมีสมาชิกคือ “อยู่ในห้องครัว” อย่างชัดเจน คือ สถานะของเขเป็นได้ทั้ง “อยู่ในห้องครัว” หรือ “ไม่อยู่ในห้องครัว” ก็ได้ ในกรณีที่เขาไปยืนระหว่างประตูที่เชื่อมทั้ง 2 ห้องเข้าด้วยกัน ทำให้เกิดสมาชิกในเซตที่มีความกำกวม คือ เขา “มีบางส่วนอยู่ในห้องครัว” ถ้ามีเพียงแค่นี้เท่าของเขอยู่ในห้องรับประทานอาหาร เราอาจกล่าวได้ว่า เขา “อยู่ในห้องครัว” 0.99 ส่วน คือ ไม่มีข้อสรุปที่แน่นอนว่า Bob “อยู่ในห้องครัว” หรือ “ไม่อยู่ในห้องครัว” ตรรกะที่เขาที่ยืนอยู่ที่ประตู

โดยปกติ Fuzzy logic จะอนุญาตให้สมาชิกในเซตมีค่าอยู่ในช่วงระหว่าง 0 ถึง 1 และมีรูปแบบทางภาษาประกอบด้วยคำที่มีหมายหมายไม่ชัดเจน เช่น slightly, quite และ very และเป็นเพียงสมาชิกบางส่วนในเซตเท่านั้น

ในการประยุกต์เอา Fuzzy Logic ไปใช้ มักจะนิยมใช้กฎ IF/THEN rules เหมือนอย่างเช่น fuzzy associative matrices กฎโดยทั่วไปสามารถแสดงได้ดังนี้

**IF variable IS set THEN action**

ยกตัวอย่างเช่นระบบการปรับระดับแรงลมของเครื่องปรับอากาศดังนี้

**IF temperature IS very cold THEN stop fan**

**IF temperature IS cold THEN turn down fan**

**IF temperature IS normal THEN maintain level**

**IF temperature IS hot THEN speed up fan**

จะสังเกตได้ว่าไม่มีคำว่า "ELSE" อยู่เลยเพราะว่า temperature อาจเป็นทั้ง "cold" และ "normal" ในเวลาเดียวกันจึงยากที่จะแบ่งออกเป็นกรณี

ใน AND, OR หรือ NOT ก็เป็น logic ที่มีอยู่ใน fuzzy logic นิยมใช้เพื่อหา minimum, maximum และ complement แล้วเมื่อมีค่าทุกค่าที่คำนวณแล้วก็จะถูกเรียกโดย Zadeh operators เพราะว่าพวกมันถูกกำหนดลงใน Zadeh's original papers สำหรับ fuzzy logic จะมี variables  $x$  และ  $y$  ดังนี้

**NOT  $x = (1 - \text{truth}(x))$**

**$x \text{ AND } y = \text{minimum}(\text{truth}(x), \text{truth}(y))$**

**$x \text{ OR } y = \text{maximum}(\text{truth}(x), \text{truth}(y))$**

ยังมี operators อื่นๆที่เป็นภาษาของมนุษย์คือคำบุพบทเช่นคำว่า "very" หรือ "somewhat" ซึ่งเราสามารถแปลงความหมายของคำเหล่านี้โดยใช้สมการทางคณิตศาสตร์

ในการเขียนโปรแกรม , ภาษา ProLog ถือเป็นเครื่องมือที่ดีในการ implement fuzzy logic ด้วยความสะดวกในการสร้าง database ของ "rules" หรือกฎเกณฑ์ต่างๆในการค้นหาค่าของ logic ให้ตรงกับ rule นั้นๆ การเขียนโปรแกรมประเภทนี้เรียกว่า logic programming ตัวอย่างอื่นๆ ถ้าผู้ชายสูงกว่าหรือเท่ากับ 1.8 เมตร , พิจารณาให้เขาเป็นคนตัวสูง

**IF male IS true AND height  $\geq 1.8$  THEN is\_tall IS true**

**IF male IS true AND height  $\geq 1.8$  THEN is\_short IS false**

Fuzzy rules เป็น logic แบบ "Imprecise logic" จึงมีความยืดหยุ่นได้ระหว่าง tall และ short

**IF height  $\geq$  medium male THEN is\_short IS agree somehow**

**IF height  $\geq$  medium male THEN is\_tall IS agree somehow**

ตัวอย่างของ fuzzy values :

**dwarf male = [0, 1.3] m**

**small male = (1.3, 1.5]**

**medium male = (1.5, 1.8]**

**tall male = (1.8, 2.0]**

**giant male > 2.0 m**

สำหรับผลที่ได้จะไม่ได้มีแค่เพียง 2 ค่าเท่านั้น (ถูกหรือผิด, 1 หรือ 2) แต่มีถึง 5 ค่าดังนี้

**agree not = 0**

**agree little = 1**

**agree somehow = 2**

**agree alot = 3**

**agree fully = 4**

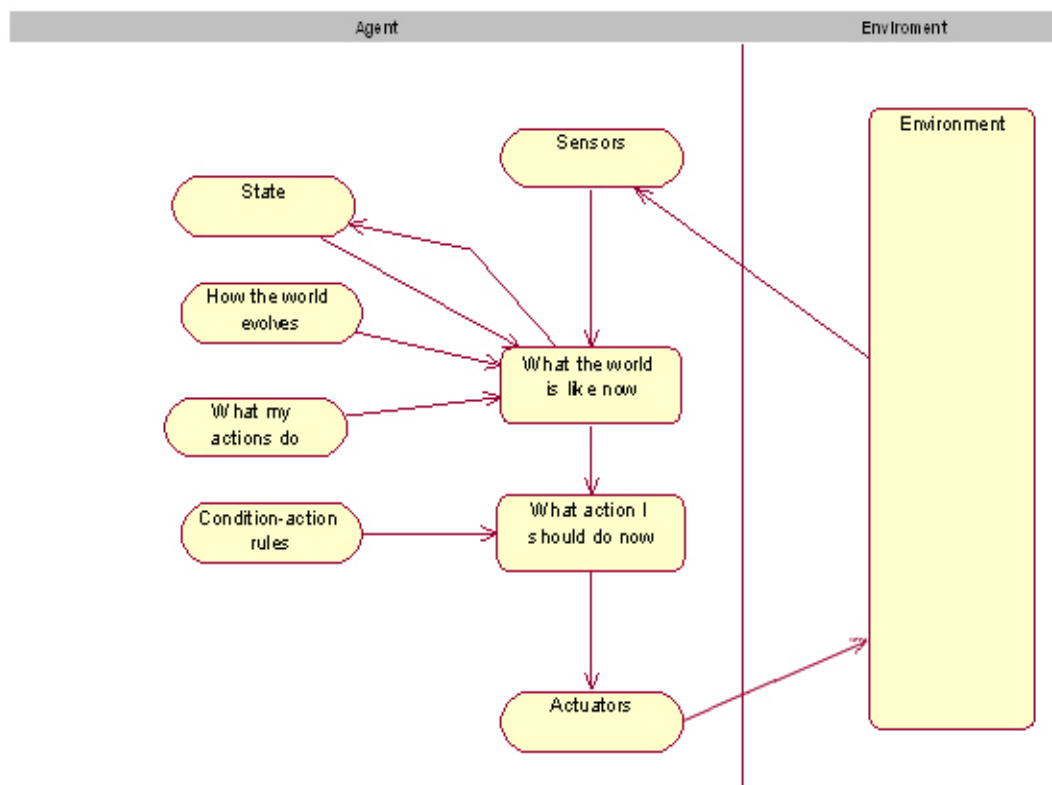
## 2.4 ทฤษฎีโครงสร้างเอเจนต์ (Structure of Agent)

**เอเจนต์แบบจดจำตอบสนองทันที (Model-Based reflex Agent)**

เป็นเอเจนต์ที่มีเฉพาะความสามารถในการรับรู้และจดจำได้แต่่าไม่มีความสามารถนำในการคิดหาข้อมูลย้อนหลังย้อนหน้าได้ (not backward forward chaining) เช่นกัน โดย

1. เอเจนต์ประเภทนี้จะมีเซนเซอร์ (Sensors) ของตนเองใช้เพื่อการรับรู้ (percept) คู่กับ การกระทำ (action) ที่ตนได้กระทำไว้เพื่อจับการเปลี่ยนแปลงจากภายนอกเพื่อคาดเดาสภาพของสิ่งแวดล้อม (Environment)
2. จากนั้นนำสภาพของสิ่งแวดล้อม (Environment) ที่ได้รับการจำลองขึ้นมา ก่อนหน้านั้น มาเทียบกับกฎเพื่อพิจารณาว่าต้องกระทำ (Action) อะไร
3. จากนั้นจึงนำมาเทียบกับกฎเพื่อพิจารณาอีกครั้งว่าต้องกระทำ (Action) อะไร





รูปที่ 2.9 เอเจนต์แบบจดจำตอบสนองทันที (Model-Based reflex Agent)